TP4 : Cryptography

Houda Toudali
Information Security

Objective

The goal of this TP was to work with classical ciphers and see how they can be broken with simple methods. I used small Python programs for Caesar, substitution and Vigenere, then kept the terminal results during each step.

Caesar Cipher

I started with the Caesar cipher using a key of 4. Each letter is shifted by the same value and the spaces are kept unchanged.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 caesar_encrypt.py
Plaintext: CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ
Encrypted: GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD
```

Caesar encryption

For decryption, I used the opposite shift. With the given key 8, the program recovered the plaintext directly.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 caesar_decrypt.py
Ciphertext: GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA
Decrypted : YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS
```

Caesar decryption

When the key was missing, I tried every possible Caesar key from 0 to 25. Key 11 was the readable result: `THE KEY IS UNKNOWN TRY BRUTE FORCE`. This shows why brute force is easy for Caesar.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 caesar_bruteforce.
Key 0 : ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP
Key 1 : DRO UOI SC EXUXYGX DBI LBEDO PYBMO
Key 2 : CQN TNH RB DWTWXFW CAH KADCN OXALN
Key 3 : BPM SMG QA CVSVWEV BZG JZCBM NWZKM
Key 4 : AOL RLF PZ BURUVDU AYF IYBAL MVYJL
Key 5 : ZNK QKE OY ATQTUCT ZXE HXAZK LUXIK
Key 6 : YMJ PJD NX ZSPSTBS YWD GWZYJ KTWHJ
Key 7 : XLI OIC MW YRORSAR XVC FVYXI JSVGI
Key 8 : WKH NHB LV XQNQRZQ WUB EUXWH IRUFH
Key 9 : VJG MGA KU WPMPQYP VTA DTWVG HQTEG
Key 10 : UIF LFZ JT VOLOPXO USZ CSVUF GPSDF
Key 11 : THE KEY IS UNKNOWN TRY BRUTE FORCE
Key 12 : SGD JDX HR TMJMNVM SQX AQTSD ENQBD
Key 13 : RFC ICW GQ SLILMUL RPW ZPSRC DMPAC
Key 14 : QEB HBV FP RKHKLTK QOV YORQB CLOZB
Key 15 : PDA GAU EO QJGJKSJ PNU XNQPA BKNYA
Key 16 : OCZ FZT DN PIFIJRI OMT WMPOZ AJMXZ
Key 17 : NBY EYS CM OHEHIQH NLS VLONY ZILWY
Key 18 : MAX DXR BL NGDGHPG MKR UKNMX YHKVX
Key 19 : LZW CWQ AK MFCFGOF LJQ TJMLW XGJUW
Key 20 : KYV BVP ZJ LEBEFNE KIP SILKV WFITV
Key 21 : JXU AUO YI KDADEMD JHO RHKJU VEHSU
Key 22 : IWT ZTN XH JCZCDLC IGN QGJIT UDGRT
Key 23 : HVS YSM WG IBYBCKB HFM PFIHS TCFQS
Key 24 : GUR XRL VF HAXABJA GEL OEHGR SBEPR
Key 25 : FTQ WQK UE GZWZAIZ FDK NDGFQ RADOQ
```

Caesar brute force

Substitution Cipher

For the substitution part, the alphabet was replaced by a fixed cipher alphabet. I used the position of each letter in the normal alphabet to find its corresponding encrypted letter.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 substitution_encrypt.py
Plaintext : KEEP THE SECRET SAFE
Encrypted : LTTN EKT YTJRTE YQGT
```

Substitution encryption

The reverse mapping was used for decryption. The given ciphertext produced `TRUST YOUR LOGIC`.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 substitution_decrypt.py
Plaintext: ERVYE DXVR SXFPJ
Decrypted : TRUST YOUR LOGIC
```

Substitution decryption

For the intercepted message, brute force is not practical because a substitution key can be any permutation of the alphabet. I counted the letters and sorted them by frequency. The most frequent symbol was `T`, followed by `E` and `R`.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 frequency_analysis.py
T 34 17.26 %
E 19 9.64 %
R 18 9.14 %
Y 16 8.12 %
X 16 8.12 %
Q 12 6.09 %
K 11 5.58 %
O 9 4.57 %
Z 8 4.06 %
P 7 3.55 %
F 6 3.05 %
S 5 2.54 %
J 4 2.03 %
B 4 2.03 %
N 4 2.03 %
V 4 2.03 %
D 4 2.03 %
H 4 2.03 %
I 4 2.03 %
G 3 1.52 %
L 3 1.52 %
C 1 0.51 %
M 1 0.51 %
```

Letter frequency

I used the frequency order only as a first guess. After that, repeated words and visible patterns helped complete more of the mapping. The partial text already showed forms such as `THE SE_RET _ESSAGE` and `LETTER _RE__EN__`.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 partial_decrypt.py
THE SE_RET _ESSAGE IS HIDDEN IN THIS _ARAGRA_H.
LETTER _RE__EN__ DOES NOT RE_EAL THE _HOLE _E_,
__T IT GI_ES A GOOD START.
LOO_ _OR _O__ON _ORDS, RE_EATED _ATTERNS,
AND SHORT _ORDS.
_HEN _O_ RE_O_ER THE _ESSAGE,
THE _IGENERE _E_ HAS _O_R LETTERS.
```

Partial substitution result

After completing the remaining letters from the word patterns, the message gave the clue: `THE VIGENERE KEY HAS FOUR LETTERS`.

Vigenere Cipher

Since the key length was 4, I separated the ciphertext into four groups according to the character position. Each group behaves like a Caesar cipher. Assuming that the most frequent plaintext letter is `E`, the four groups gave the key letters `C`, `O`, `D` and `E`.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 vigenere_key.py
Group 1
Most frequent letter: G
Key letter: C

Group 2
Most frequent letter: S
Key letter: O

Group 3
Most frequent letter: H
Key letter: D

Group 4
Most frequent letter: I
Key letter: E
```

Vigenere key recovery

I then repeated the key `CODE` over the ciphertext and subtracted the corresponding shifts. The final message was recovered and ended with the access word `ECLIPSE`.

```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 4/code$ python3 vigenere_decrypt.py
Key: CODE
Decrypted message:
THESECRETKEEPERLEFTTHESENOTESBECAUSETHELETTEREAPPEARSEVERYWHEREEVERYTEAMMEMBERSHOULDCHECKEACHCOLUMNSEEKTHEMOSTFREQUENTSYMBOLANDUSEITTORECOVERTHEKEYWHENTHEME
SSAGEBECOMESCLEARENTERTHEACCESSWORDECLIPSE
```

Final Vigenere decryption

Conclusion

The Caesar cipher was easy to brute force because its key space is very small. The substitution cipher needed frequency analysis plus word patterns, while Vigenere became easier once the key length was known. The final access word was `ECLIPSE`.